SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
 <input type="text" name="name">
 <input type="email" name="email">
 <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure. 2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
 <head>
 <title>Company</title>
 </head>
 <body>
 <div>
 <h1>Welcome
 <p>About us
 </div>
 </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
 <input type="text" name="username">
 <input type="password" name="password">
 <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

*I am **Nirosha M (192311411)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:Nirosha.M

RUBRICS FOR CALCULATION:

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understandi ng of Concepts	24	Demonstrates thorough, accurate understanding of concepts	Good understandi ng with minor gaps	Basic understandi ng with some misconcepti on s	Poor understan din g; major misconce ptio ns
Explanation	23	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	19	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	14	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organizatio n, incoherent answers
Accuracy	9	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	4	Clear, neat, professional presentation	Understand able with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

TOTAL=93/100

1. Online Symposium Portal

1.1 Identify appropriate HTML5 semantic elements missing in the structure.

The given HTML form lacks several important HTML5 semantic elements that improve webpage structure, accessibility, and readability.

- <header> – Defines the introductory content or heading of the webpage.
- <main> – Represents the main content of the webpage.
- <section> – Groups related content, such as symposium registration details.
- <footer> – Contains footer information, such as copyright details.
- <label> – Provides descriptive labels for form input fields.

1.2 Modify the structure to make it standards-compliant.

The following HTML5 code provides a semantic and standards-compliant structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Online Symposium Registration</title>
</head>
<body>
   <header>
      <h1>Online Symposium Portal</h1>
   </header>

   <main>
      <section aria-labelledby="registration-heading">
         <h2 id="registration-heading">Student Registration</h2>

         <form action="/register" method="POST">
            <label for="name">Full Name:</label>
            <input type="text" id="name" name="name"
                  autocomplete="name" required>

            <label for="email">Email Address:</label>
            <input type="email" id="email" name="email"
                  autocomplete="email" required>
```

```
            <button type="submit">Submit</button>
        </form>
    </section>
</main>

<footer>
    <p>&copy; 2026 Online Symposium Portal</p>
</footer>
</body>
</html>
```

Explanation: The document uses the HTML5 doctype, language declaration, character encoding, and viewport settings. Semantic elements provide a clear page structure, while labels and required fields improve accessibility and validation.

1.3 Interpret the HTTP request generated during form submission.

When a student submits the form, the browser generates an HTTP POST request to the /register endpoint.

Example request:

```
POST /register HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

name=Arun&email=arun%40example.com
```

The request contains the following components:

1. HTTP method: POST sends the form data in the request body.
2. Request URL: /register identifies the server endpoint.
3. Headers: Provide information such as the host and content type.
4. Request body: Contains the submitted name and email in URL-encoded form.

The browser encodes special characters appropriately before transmitting the data. POST is suitable for registration because the submitted information is placed in the request body rather than appended to the URL.

1.4 Analyze the HTTP response cycle from server to browser.

The HTTP response cycle involves these steps:

1. The browser sends the POST request to the web server.
2. The server receives and validates the submitted information.

3. The server processes the registration and may store the information in a database. 4. The server returns an HTTP response with a status code, headers, and an optional response body.
5. The browser interprets the response and displays the appropriate page or message.

Common HTTP response status codes include:

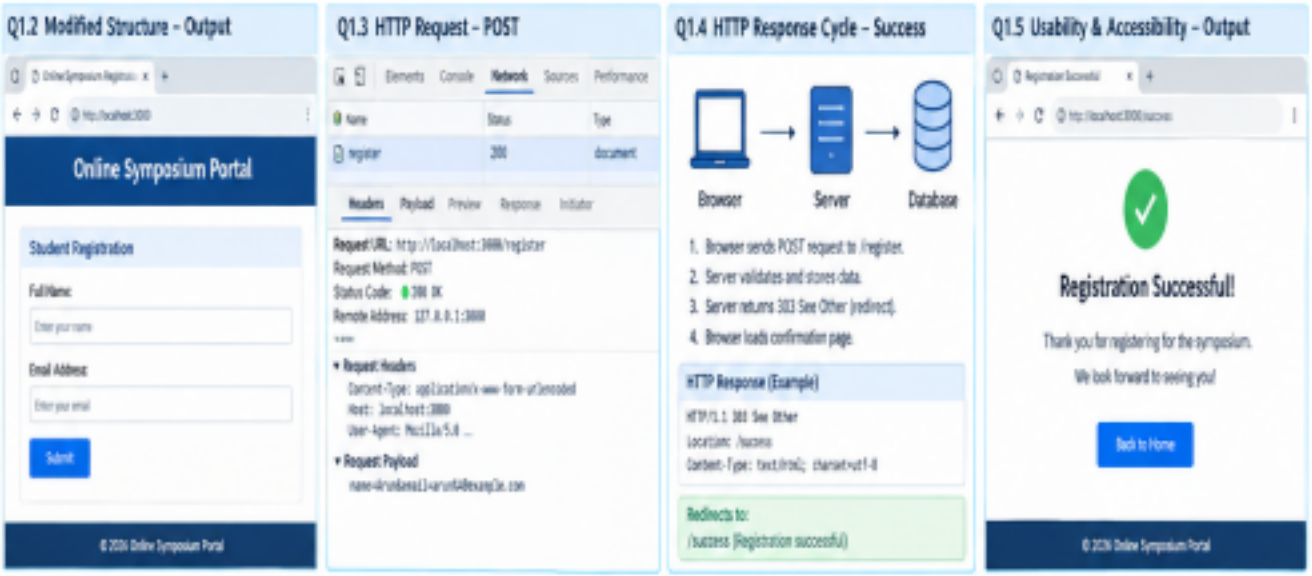
- 200 OK – The request was processed successfully.
- 201 Created – A registration resource was created successfully.
- 400 Bad Request – The submitted data is invalid.
- 500 Internal Server Error – An unexpected server-side error occurred.

For a successful form submission, the server may return 303 See Other with a Location header to redirect the browser to a confirmation page. This prevents accidental duplicate submissions when the user refreshes the page.

1.5 Suggest improvements for usability and accessibility.

The following improvements can make the portal easier and safer to use:

1. Add visible labels for all form fields and ensure keyboard navigation works correctly. 2. Use HTML5 validation attributes such as required, type="email", and suitable input constraints. 3. Display clear success messages and meaningful error messages near the relevant fields. 4. Make the webpage responsive using the viewport meta tag and responsive CSS.
5. Use HTTPS to encrypt information transmitted between the browser and server.
6. Maintain sufficient colour contrast, visible keyboard focus, and appropriate heading order.
7. Provide server-side validation because client-side validation can be bypassed.



2. Cross-Browser Compatibility Issue

2.1 Identify improper nesting and missing closing tags.

The given HTML contains several structural problems:

- The <h1> element is not properly closed.
- The <p> element is not properly closed.
- The heading and paragraph are nested incorrectly because the heading begins before the paragraph without being closed.
- The document is missing the <!DOCTYPE html> declaration.
- The <html> element does not specify a language.
- The <head> section lacks character encoding and viewport metadata.
- Appropriate semantic elements such as <header>, <main>, and <section> are missing.

These issues can cause browsers to interpret and display the document differently.

2.2 Analyze how different browsers may interpret this structure.

Browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari use HTML parsing algorithms to interpret markup.

When invalid nesting or missing closing tags occurs:

1. Browsers may automatically insert missing elements or close certain elements during parsing. 2. The resulting Document Object Model (DOM) may differ from the structure intended by the developer. 3. CSS selectors may behave unexpectedly if the resulting DOM differs from the expected structure. 4. Text formatting, spacing, and element positioning may vary because of differences in default browser styles. 5. Accessibility tools may encounter a confusing heading hierarchy or document structure.

Although modern browsers attempt to recover from malformed HTML, relying on automatic correction is not recommended.

2.3 Identify missing semantic elements.

The following semantic elements can improve the structure:

- <header> – Contains the company name and introductory heading.
- <main> – Identifies the primary page content.
- <section> – Groups related information, such as the About Us content.
- <footer> – Contains copyright information or contact details.

Semantic elements make the document easier to maintain and help assistive technologies understand the page structure.

2.4 Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company Website</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>

  <main>
    <section aria-labelledby="about-heading">
      <h2 id="about-heading">About Us</h2>
      <p>Learn more about our company and services.</p>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Company Website</p>
  </footer>
</body>
</html>
```

Explanation: The corrected document includes a valid doctype, properly nested and closed elements, semantic page sections, character encoding, and viewport metadata. These practices improve standards compliance and browser consistency.

2.5 Suggest techniques to ensure cross-browser compatibility.

The following techniques should be used:

1. Validate HTML: Use the W3C Markup Validation Service to identify structural errors.
2. Use CSS resets or normalization: Reduce inconsistencies in default browser styles. 3.
Follow web standards: Use valid HTML5 and CSS3 syntax.
4. Test across browsers: Verify the website in Chrome, Firefox, Edge, and Safari.
5. Use responsive design: Apply CSS media queries and flexible layouts.
6. Avoid unsupported features: Check browser support before using new HTML, CSS, or JavaScript

features.

7. Perform accessibility testing: Verify keyboard navigation, heading hierarchy, and screen-reader compatibility.
8. Use automated testing tools: Tools such as Playwright can automate browser-based layout and functionality checks.

These techniques help maintain consistent layout, functionality, and accessibility across different browsers and devices.



3. Form Submission Failure

3.1 Identify issues in the form configuration.

The given code is:

```
<form action="/submit" method="GET">
    <input type="text" name="username">
    <input type="password" name="password">
    <button type="button">Login</button>
</form>
```

The main issues are:

1. Incorrect button type: The button uses type="button", which does not submit the form by default. 2. Inappropriate HTTP method: GET is unsuitable for transmitting passwords because submitted values are appended to the URL.
3. Missing labels: The input fields lack associated labels, reducing accessibility.
4. Missing validation: The fields do not use the required attribute.
5. Missing security considerations: Authentication must be performed over HTTPS, with server-side validation and secure session handling.

3.2 Analyze why the HTTP request is not triggered.

The primary reason is the button declaration:

<button type="button">Login</button>

A button with type="button" performs no form-submission action unless JavaScript explicitly assigns one.

Therefore, clicking Login does not trigger the browser's normal form-submission process, and the server does not receive a request from that click.

Changing the button to type="submit" enables normal form submission, provided that validation succeeds and no other script prevents submission.

3.3 Interpret the role of the GET method in this scenario.

The GET method requests a resource from the server. When used with a form, the browser normally appends successful form controls to the URL as query parameters.

For example:
GET /submit?username=Arun&password=abc123 HTTP/1.1
Host: example.com

The example illustrates why GET is inappropriate for login credentials.

The disadvantages include:

- Sensitive values may appear in browser history, server logs, and URLs shared or copied by users.
- URLs may be stored in monitoring systems or analytics records.
- GET requests are intended primarily for retrieving resources, not performing authentication-related state changes.

GET can be useful for non-sensitive searches and filters. For login forms, POST over HTTPS is the appropriate choice.

Important: POST alone does not encrypt information. HTTPS is required to protect credentials during transmission.

3.4 Propose a corrected implementation.

<!DOCTYPE html>
<html lang="en">
<head>

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login Page</title>
</head>
<body>
    <main>
        <h1>Login</h1>

        <form action="/submit" method="POST">
            <label for="username">Username:</label>
            <input type="text" id="username" name="username"
                autocomplete="username" required>

            <label for="password">Password:</label>
            <input type="password" id="password" name="password"
                autocomplete="current-password" required>

            <button type="submit">Login</button>
        </form>
    </main>
</body>
</html>
```

Explanation: The corrected implementation uses POST, a submit button, associated labels, and required fields. The browser submits the form to /submit when the user clicks Login and validation succeeds. The server must then authenticate the credentials and return an appropriate response.

3.5 Suggest improvements for secure data transmission.

The following measures should be implemented to protect user credentials:

1. Use HTTPS: Encrypt communication between the client and server using TLS.
2. Use POST for authentication: Avoid placing passwords in query strings.
3. Hash passwords securely: Store passwords using a dedicated password-hashing algorithm such as Argon2id, bcrypt, or scrypt, with appropriate parameters and salts.
4. Validate input on the server: Never rely exclusively on browser-side validation.
5. Protect sessions: Use secure session cookies with Secure, HttpOnly, and an appropriate SameSite setting.
6. Prevent brute-force attacks: Apply rate limiting and appropriate authentication controls. 7. Avoid logging secrets: Never log plaintext passwords or include them in error messages. 8. Use CSRF protection where applicable: Protect cookie-based authentication flows against cross-site request forgery.

These measures improve confidentiality, integrity, and the overall security of the authentication process.

